Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: D. Kanishkarane
Roll No: 24011101021

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T\mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset
Source: UCI Machine Learning Repository
Download Link:
`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

1 6. Source Code

GitHub Repository

7. Experimental Results

7.1 Training Summary

Parameter	Value
Dataset Size	1372
Train/Test Split	80% / 20%
Learning Rate	0.01
Epochs Run	100
Final Weights	[−0.223, −0.269, −0.2465, −0.0044]
Final Bias	−0.11
Accuracy	98.55%
Precision	96.83%
Recall	100.00%
F1-score	98.39%

Table 1: Training Summary

7.2 Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
20	15	−0.1449	−0.1969	−0.1735	−0.0212	−0.08
40	16	−0.1889	−0.2131	−0.1914	−0.0217	−0.09
60	11	−0.2043	−0.2335	−0.2175	−0.025	−0.11
80	12	−0.2145	−0.2559	−0.2385	−0.0165	−0.11
100	18	−0.223	−0.269	−0.2465	−0.0044	−0.11

Table 2: Epoch-wise Learning (selected epochs)

7.3 Evaluation Metrics

The trained perceptron was evaluated on the held-out test set (20% of the data, 275 samples), yielding the following results:

Metric	Value
Accuracy	98.55%
Precision	96.83%
Recall	100.00%
F1-score	98.39%

Table 3: Test Set Evaluation Metrics

7.4 Confusion Matrix

	Predicted Authentic (0)	Predicted Forged (1)
Actual Authentic (0)	149	4
Actual Forged (1)	0	122

Table 4: Confusion Matrix on Test Set

The model correctly classified 271 out of 275 test samples. All 4 errors were false positives (authentic notes misclassified as forged); there were no false negatives, meaning every forged note in the test set was correctly identified.

7.5 Comparison with Scikit-learn's Perceptron

Metric	From-Scratch Perceptron	Scikit-learn Perceptron
Accuracy	98.55%	96.73%
Precision	96.83%	94.49%
Recall	100.00%	98.36%
F1-score	98.39%	96.39%

Table 5: From-Scratch vs Scikit-learn Perceptron

8. Generated plots with Interpretation

i) Feature Histograms

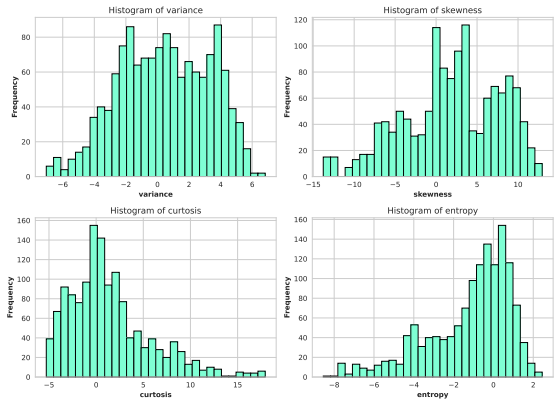


Figure 1: Feature Histograms of the dataset features.

The variance and skewness features show large,multi-modal distributions which states that only they contain information which is meaningful for separating the two classes with relative strong correlation with class label(-0.725,-0.445 respectively).While curtosis and entropy are skewed with long tails on both sides and proves weaker correlation with class(0.156 and -0.023),which clearly shows that they are not discriminative for this task.

ii) Correlation Heatmap

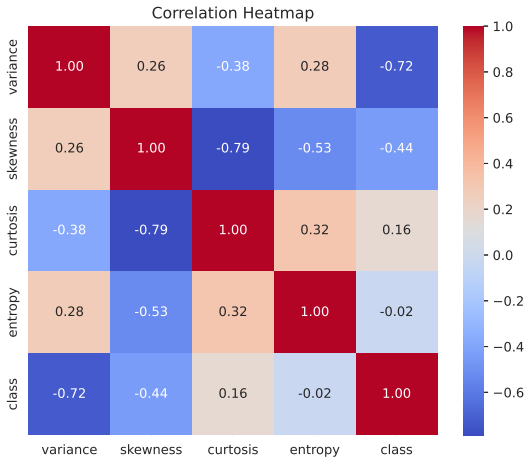


Figure 2: Correlation Heatmap

The figure shows that variance is the only feature which is strongly associated with the target class and has a correlation of -0.72 and skewness which has correlation of -0.44.Curtosis and entropy show much weaker relationships with class (0.16 and -0.02),they are individually not useful for differentiating each feature(authentic and forged).To further add on,curtosis and skewness are strongly correlated with each other(-0.79) which is shown through the overlap of information.

iii) Scatter Plot

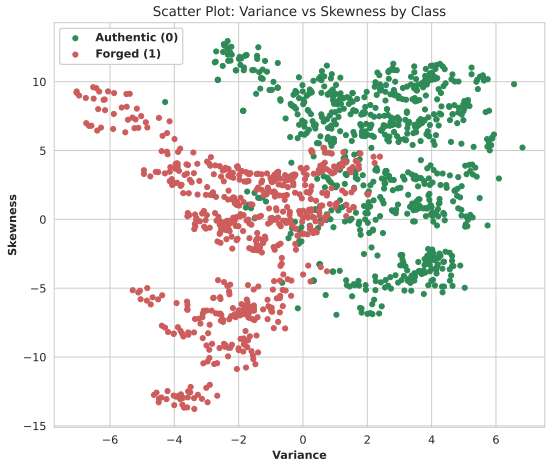


Figure 3: Scatter Plot

From the figure,we can see that there is clustering along the variance axis.The red circled(Forged) are concentrated in the left side while the green circled(authentic) dominate the right side.Therefore, we can see a obvious overlap around variance -1 to 2 and in this region the decision boundary tends to misclassify.

iv) Training Error vs Epoch

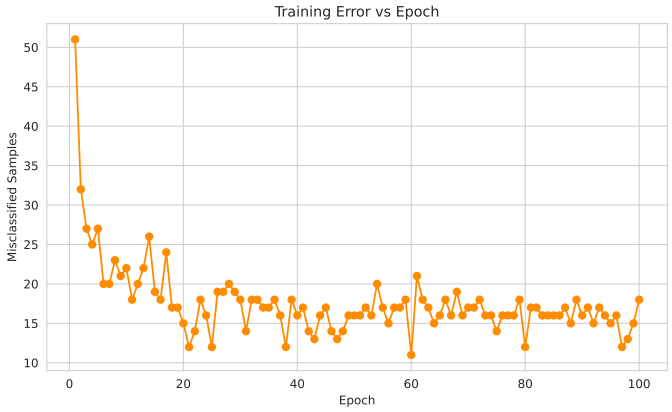


Figure 4: Training Error vs Epoch

The count of misclassification starts from 51 at epoch 1 and drops down to a very less number within the first 15-20 epochs,after which it gets flattened and revolves between 12-21 errors.This shows that the dataset is not entirely linearly separable in the entire feature space- there is a small subset of points which constantly gets misclassified,these points lie close to the decision boundary.Even then the accuracy is good because the misclassification is only borderline.

v) Weight Evolution

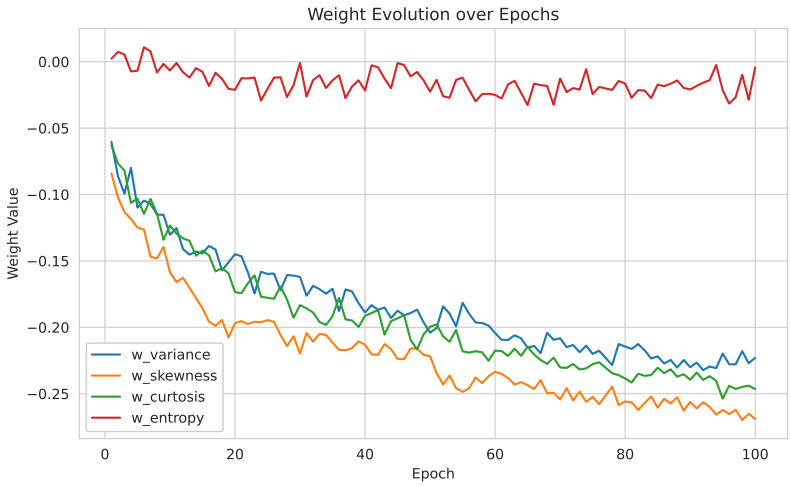


Figure 5: Weight Evolution During Training

Three of the four weights - that weights of variance,skewness and curtosis all of them steadily move downward with skewness ending up being the most negative(-0.27), and variance and curtosis has nearly close values(-0.22 -0.24),Entropy alone stays flat.Curtosis weights grows as largely as variance and indicates correlation with skewness(-0.79).Entropy barely contributes to the model's classification.

vi) Bias Evolution

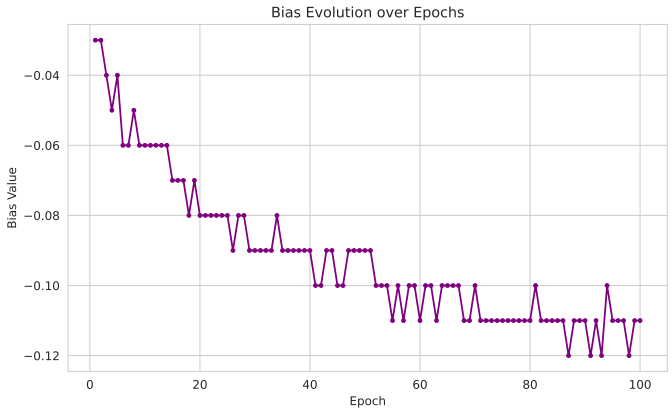


Figure 6: Bias Evolution During Training

The bias term starts at 0 and keeps decreasing until epoch 100,which shows that the models keeping changing the position of decision boundary as it keeps learning.The decline does not happen smoothly and instead happens in plateaus which suggests the model is not completely stable - where most are classfied correctly and some fail.The weight updates keep reducing over time - by epoch

50 it almost reaches convergence.Overall, the negative bias shows we need more evidence before classifying it as authentic.

vii) Confusion Matrix

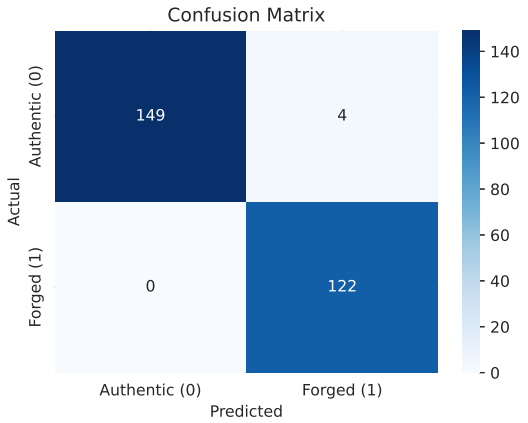


Figure 7: Confusion Matrix

The model classifies correctly about 149 out of 153 authentic notes and all 122 forged notes misclassifying only 4 authentic notes as forged and none of the forged notes as authentic.Hence the accuracy is about 0.98 and recall 1.00 on detecting forged notes.Precision is around 0.96 showing only a small number of authentic notes getting misclassified.The absence of false negatives is a notable point here because failing is more costly.The figure indicates the classifier which is reliable upto an good extent with a bias towards minimising forged ones.

viii) Learning Rate Comparison

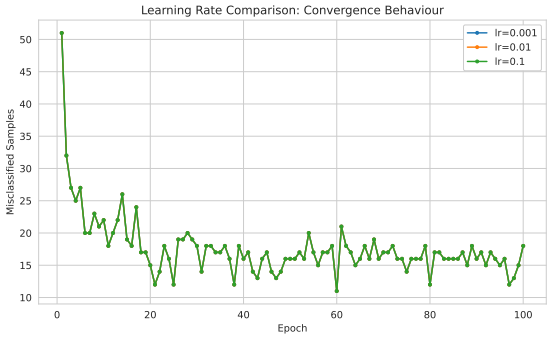


Figure 8: Learning Rate Comparison

All the learning rates have given identical curves and hence only one curve appears in the images as the others are overlapped.So , it indicates convergence is not dependent on learning rate.This is expected outcome for a perceptron,where the learning rate does not indicate the direction of weight update.Misclassification drops from 51 to 20 and shows fast learning initially.The data is

not entirely linearly separable.Hence tuning the learning rate alone will not the improve the model's performance.

ix) Perceptron Decision Boundary

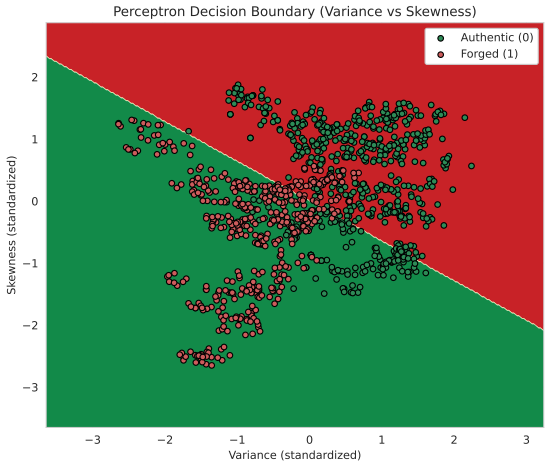


Figure 9: Perceptron Decision Boundary (Variance vs Skewness)

The figure visualises how our model-perceptron separated the authentic and forged notes using the variance and skewness features.The linear decision boundary splits the region into two halves - the red(forged notes) and green(authentic notes).Most green samples are in the right corner and red samples are in lower left corner.These indicates that the classes can be separated well.However there is a noticable overlap near the boundary.This confirms that a purely linear boundary using only two features achieves good but imperfect separation, motivating the use of additional features (kurtosis and entropy) for the full model to improve classification accuracy.

x) Box Plots

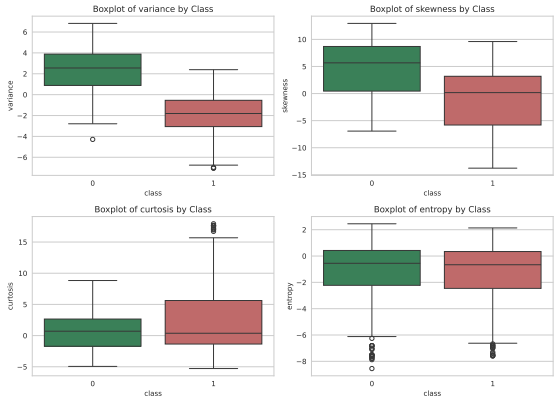


Figure 10: Perceptron Decision Boundary (Variance vs Skewness)

The figures shows the distribution of the features - variance,curtosis,skewness and entropy.Variance shows the least overlap and helps us distinguish.Entropy shows the highest overlap and hence aids least in distinguishing.

9. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80/20
Learning Rate	0.01
Epochs	100
Final Weights	[-0.223, -0.269, -0.2465, -0.0044]
Final Bias	-0.11
Accuracy	0.9855
Precision	0.9683
Recall	1.0
F1-score	0.9839

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
20	15	-0.1449	-0.1969	-0.1735	-0.0212	-0.08
40	16	-0.1889	-0.2131	-0.1914	-0.0217	-0.09
60	11	-0.2043	-0.2335	-0.2175	-0.025	-0.11
80	12	-0.2145	-0.2559	-0.2385	-0.0165	-0.11
100	18	-0.223	-0.269	-0.2465	-0.0044	-0.11

10. Additional Tasks

1. **Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.** Deep networks are trained using gradient descent and backpropagation , and they mainly require certain activation functions so that they can compute how much each weight contributed to the error . The derivative of the step function is zero everywhere (undefined at $z = 0$), so gradients cannot flow backwards through it.The sigmoid and later activations like ReLU give non-zero, smooth gradients. This is why the step function is restricted to simple linear classifiers like the single-layer perceptron and not used in modern deep learning models.

Aspect	Step Function	Sigmoid Function
Output	Discrete {0, 1}	Continuous (0, 1)
Differentiable	No	Yes, smooth everywhere
Gradient-based learning	Not usable	Usable — supports backpropagation
Interpretability	Hard class decision	Probabilistic confidence score
Sensitivity to weight changes	None (output flips discretely)	Smooth, gradual change

Table 6: Step vs Sigmoid Activation Function

2. **Compare the implementation with Scikit-learn's Perceptron.**

   Both the from-scratch implementation and `sklearn.linear_model.Perceptron` use the same perceptron learning rule. On the test set, the from-scratch model achieved an accuracy of 98.55%, while scikit-learn's implementation achieved 96.73% (Precision: 94.49%, Recall: 98.36%, F1: 96.39%). The small difference in results comes only due to the implementation rather than a difference in the actual algorithm: scikit-learn keeps shuffling training samples by default, which the from-scratch version does not include. This confirms that the from-scratch implementation properly reproduces the actual standard perceptron learning algorithm.

3. **Repeat the experiment using learning rates 0.001, 0.01 and 0.1.**

   Because the perceptron's weight update depends only on the *sign* of the error $(y - \hat{y})$ rather than its magnitude, the learning rate mainly scales the step size rather than changing the direction of the convergence path. On this particular dataset, almost linearly separable, all three learning rates converge to a similar final error count and test accuracy.

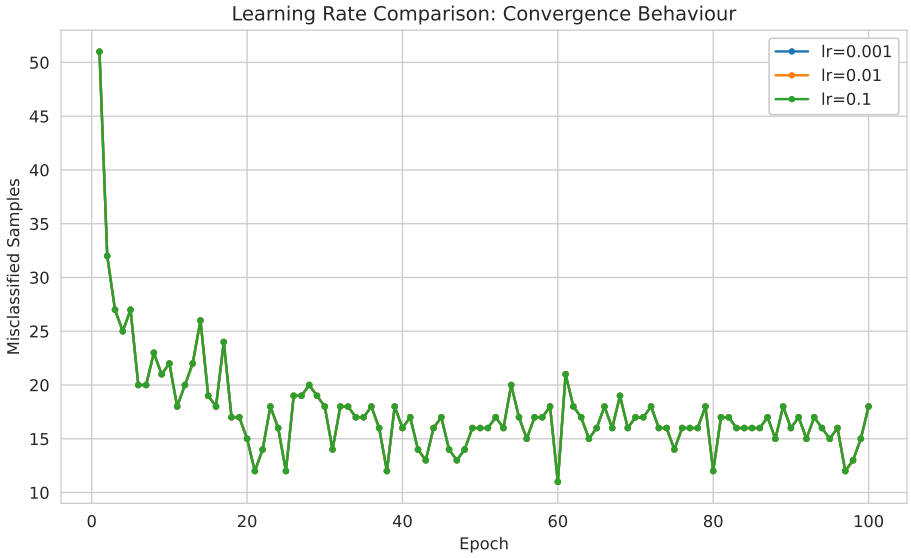


Figure 11: Learning Rate Comparison

4. **Explain why a Single Layer Perceptron cannot solve the XOR problem.**

The XOR function is not linearly separable: there is no single straight line in two-dimensional space that separates the outputs $(0,0) \to 0$ and $(1,1) \to 0$ from $(0,1) \to 1$ and $(1,0) \to 1$, because the two points belonging to class 1 sit on opposite corners from the two points belonging to class 0. A single-layer perceptron with a step activation function can only represent one linear decision boundary defined by $w^T x + b = 0$, so it cannot represent this pattern fundamentally, no matter how it is trained. This limitation led to the development of multilayer perceptrons, which combine several linear boundaries through hidden layers and non-linear activation functions, allowing the network to represent non-linearly separable functions like XOR.

5. **Study the effect of feature normalization on model convergence.**

   Because the perceptron's weight update is proportional to the input value, i.e. $w_{new} = w_{old} + \eta(y - \hat{y})x$, features with a large range would dominate the weight update compared to a feature on a small scale, even if the small-scale feature were more informative. Normalizing all four features to zero mean and unit variance, as done during preprocessing leads to faster and more stable convergence during training and prevents the learning process from being biased toward whichever feature happens to have the largest raw magnitude, which in this dataset would otherwise be `skewness`.

6. **Code the perceptron learning algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages.**

   A single-layer perceptron with a step activation function was implemented from scratch, using the standard update rule $w \leftarrow w + \eta(y - \hat{y})x$ and $b \leftarrow b + \eta(y - \hat{y})$, which was applied only when a single training sample was misclassified. The perceptron was trained to implement the four input combinations of the OR and AND gates, and on the two input values of the NOT gate, until a full pass through the data produced zero errors manually. The OR gate converged after 6 weight updates, the AND gate after 11 updates, and the NOT gate after 2 updates.

Update	**OR (w, b)**	**AND (w, b)**	**NOT (w, b)**
0 (init)	[0, 0], 0.00	[0, 0], 0.00	[0], 0.00
1	[0, 0], -1.00	[0, 0], -1.00	[-1], -1.00
2	[0, 1], 0.00	[1, 1], 0.00	[-1], 0.00 (converged)
3	[0, 1], -1.00	[1, 1], -1.00	–
4	[1, 1], 0.00 (converged)	[1, 0], -2.00	–
5	[1, 1], -1.00	[2, 1], -1.00	–
6	–	[2, 0], -2.00	–
7	–	[1, 0], -3.00	–
8	–	[2, 1], -2.00	–
9	–	[1, 1], -3.00	–
10	–	[2, 2], -2.00	–
11	–	[2, 1], -3.00 (converged)	–

Table 7: Weights and bias after each update for OR, AND, and NOT gates

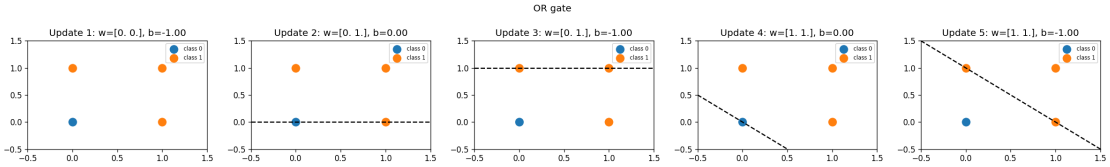


Figure 12: Decision boundary after each weight update — OR gate

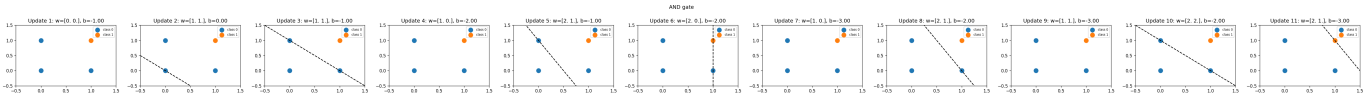


Figure 13: Decision boundary after each weight update — AND gate

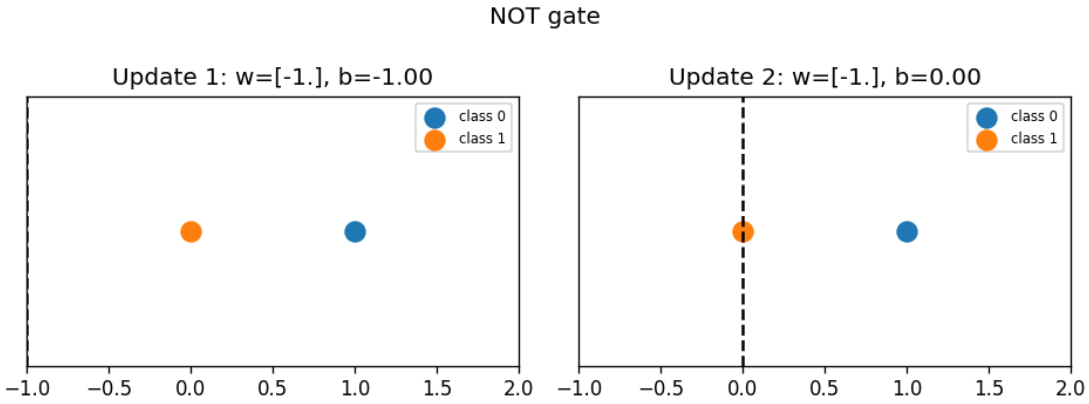


Figure 14: Decision boundary after each weight update — NOT gate

Inference: The three gates given in the question namely OR,AND and NOT are functions which are separable linearly , which means a single straight line called a decision boundary is more than enough to clearly classify both the classes.The perceptron coverges decently faster for all the three gates : each update shifts the boundary when each point is misclassified and once the separation is done properly , further updates do not take place.AND needed more updates than OR because there is only one (1,1) point and other points are all negative, so the decision boundary has to make more shifts to make the model correctly classify points.

11. Discussion

The perceptron achieved 98.55% test accuracy, but training error never reached zero, which shows that the dataset is close to, but not perfectly, linearly separable. The learned weights confirm that `variance` and `skewness` helps us take most decisions, while `entropy` contributes almost nothing.

12. Conclusion

The objective of this experiment was to implement a single-layer perceptron to classify banknotes as authentic or forged with the use of the Banknote Authentication dataset. After the data was preprocessed, the perceptron was trained using the standard perceptron learning rule. Training the

model was done several times nearly 100 epochs, and during the 100 iterations, the model showed certain signs of convergence although not fully. The number of samples misclassified converged, the weights converged, and the bias converged.

The resulting model was a success with an overall accuracy of 98.55. The resulting model also had great precision, recall, and great F1-scores. The resulting model had an almost perfect confusion matrix with no misclassified samples, and the model had perfect learning curves. This model had a great performance because the two classes of the dataset were almost fully linearly separable, which made the simple model of the perceptron an excellent choice for this task.

Even though the model was a success, it also demonstrated a significant drawback of a single-layer perceptron. This model is able to only classify data with linear separability and great difficulty in classifying data with non-linear separability. The focus of this experiment was to understand the limitations of single-layer models and also understand the challenges of multi-layer models and the need for the multi-layer feed forward networks.

13. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.